ممهور

التقرير الفني الشامل

نظام متكامل لإدارة دورة حياة المحتوى واعتماده — من الإنشاء حتى النشر

📅 تاريخ التقرير: 25 أبريل 2026 📦 الإصدار: 0.1.0 🔧 الحالة: مرحلة التطوير — MVP

فهرس المحتويات

الرؤية والهوية ①

البنية التحتية (Technology Stack) ②

قاعدة البيانات ③

نظام المصادقة والأمان ④

هيكل المشروع (Architecture) ⑤

واجهات المنصة — Screenshots ⑥

رحلة المستخدم الكاملة (User Journey) ⑦

Server Actions ⑧

نظام التصميم (Design System) ⑨

حالة ربط البيانات الحقيقية ⑩

المتغيرات البيئية ⑪

البنية التحتية للحاويات (Docker) ⑫

التوصيات والخطوات المستقبلية ⑬

كيفية التشغيل ⑭

1. الرؤية والهوية

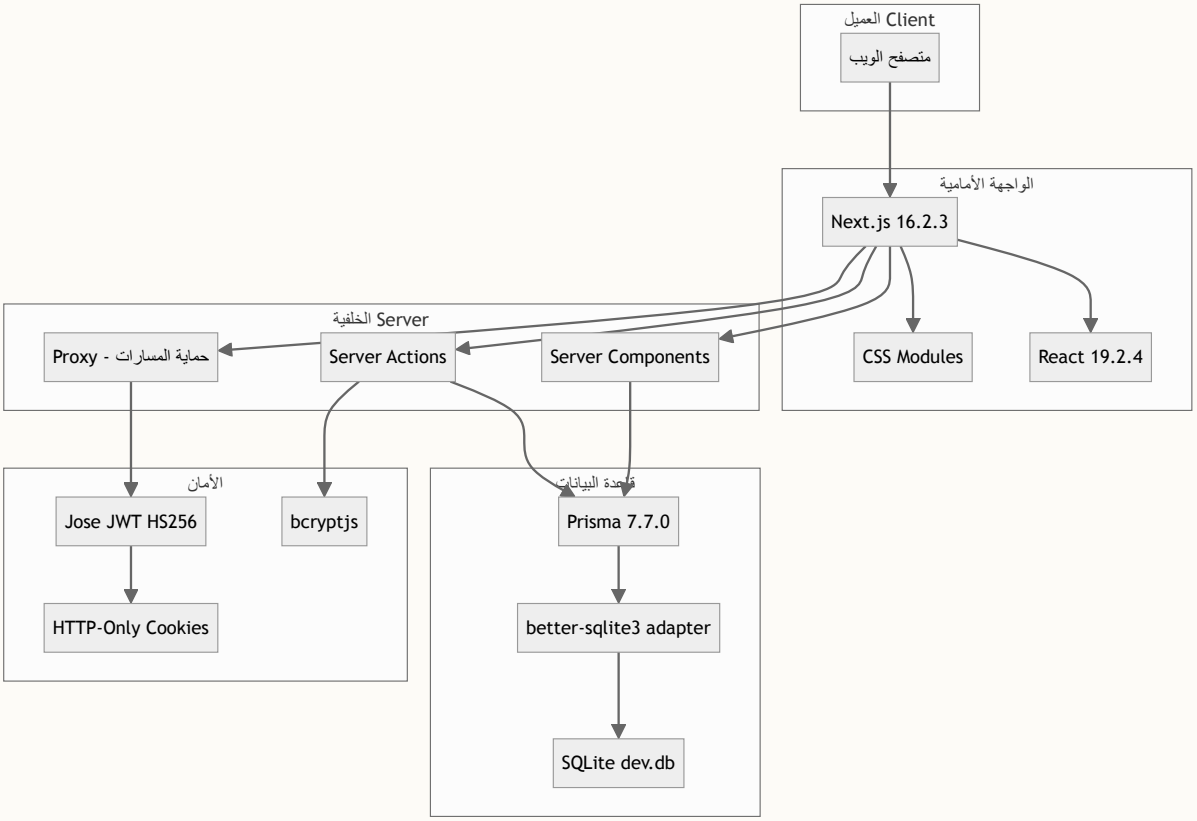
ممهور هي منصة عربية متكاملة لإدارة دورة حياة المحتوى واعتماده. الاسم مشتق من "ختم" (مَمْهور = مُختَم بالختم الرسمي) — ما يعكس الفلسفة الأساسية: **كل محتوى يمر بمراحل مراجعة واعتماد حتى يصبح "ممهوراً"**.

الهوية البصرية

العنصر	القيمة
اللون الرئيسي	`#212842` Midnight Indigo
اللون الثانوي	`#F0E7D5` Vanilla Cream
اللون المميز	`#C4A265` Gold
خط العناوين	Cairo (عربي + لاتيني)
خط النصوص	Amiri (عربي + لاتيني)
الاتجاه	RTL (من اليمين لليسار)
الشعار	ختم SVG مخصص يحتوي على حرف "م" داخل إطار دائري ذهبي

2. البنية التحتية (Technology Stack)

2.1 نظرة عامة



2.2 الاعتمادات (Dependencies)

اعتمادات الإنتاج

الحزمة	الإصدار	الوظيفة
`next`	16.2.3	إطار العمل الأساسي — App Router
`react`	19.2.4	مكتبة واجهات المستخدم
`react-dom`	19.2.4	ربط React بالمتصفح
`@prisma/client`	^7.7.0	عميل ORM لقاعدة البيانات
`@prisma/adapter-better-sqlite3`	^7.8.0	محول SQLite لـ Prisma 7
`better-sqlite3`	^12.9.0	مشغل SQLite الأصلي
`bcryptjs`	^3.0.3	تشفير كلمات المرور (salt rounds: 10)
`jose`	^6.2.2	تشفير/فك تشفير JWT
`zod`	^4.3.6	التحقق من صحة البيانات TypeSafe
`server-only`	^0.0.1	منع استيراد ملفات الخادم في العميل

اعتمادات التطوير

الحزمة	الإصدار	الوظيفة
`prisma`	^7.7.0	أداة CLI لإدارة قاعدة البيانات
`typescript`	^5	TypeScript للأنواع
`eslint`	^9	فحص جودة الشفرة
`eslint-config-next`	16.2.3	إعدادات ESLint لـ Next.js
`@types/*`	متعددة	أنواع TypeScript

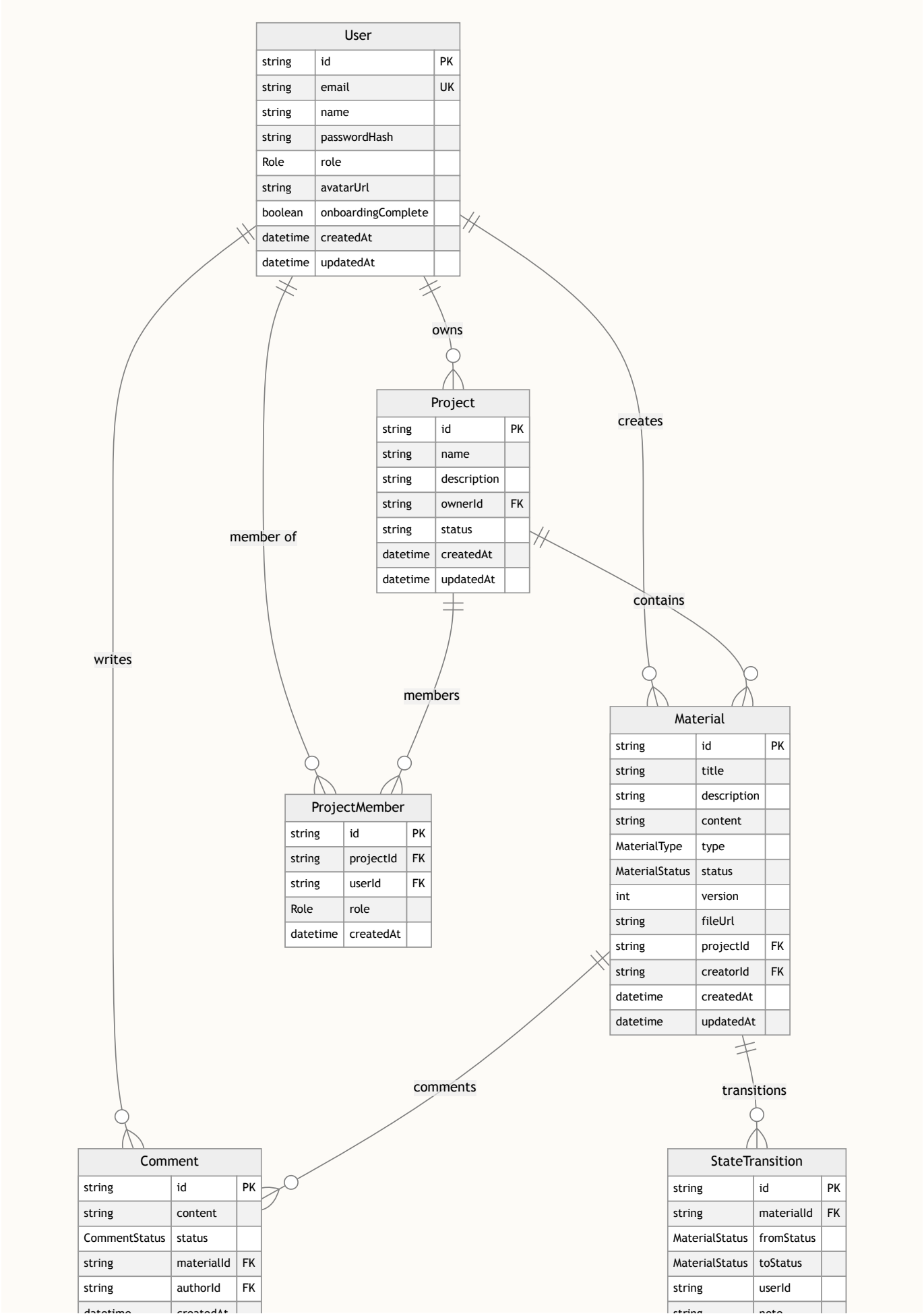
2.3 ملفات الإعداد الجذرية

الملف	الوظيفة
`package.json`	التبعيات والسكربتات
`tsconfig.json`	إعداد TypeScript — ES2017, strict, path alias @/*
`next.config.ts`	إعداد Next.js — حالياً فارغ
`prisma.config.ts`	إعداد Prisma — مسار الميجريشن ومصدر البيانات
`.env`	DATABASE_URL + SESSION_SECRET
`proxy.ts`	Middleware لحماية المسارات
`docker-compose.yml`	PostgreSQL 15 للإنتاج (غير مستخدم حالياً)

> 📝 **ملاحظة**
>
> المشروع يستخدم **SQLite** في التطوير عبر better-sqlite3. يوجد docker-compose.yml جاهز لـ PostgreSQL 15 عند الانتقال للإنتاج.

3. قاعدة البيانات

3.1 مخطط العلاقات (ER Diagram)


User
string id PK
string email UK
string name
string passwordHash
Role role
string avatarUrl
boolean onboardingComplete
datetime createdAt
datetime updatedAt
owns
Project
string id PK
string name
string description
string ownerId FK
string status
datetime createdAt
datetime updatedAt
creates
member of
contains
writes
members
ProjectMember
string id PK
string projectId FK
string userId FK
Role role
datetime createdAt
Material
string id PK
string title
string description
string content
MaterialType type
MaterialStatus status
int version
string fileUrl
string projectId FK
string creatorId FK
datetime createdAt
datetime updatedAt
comments
transitions
Comment
string id PK
string content
CommentStatus status
string materialId FK
string authorId FK
StateTransition
string id PK
string materialId FK
MaterialStatus fromStatus
MaterialStatus toStatus
string userId

datetime	updatedAt	

datetime	createdAt	

3.2 التعدادات (Enums)

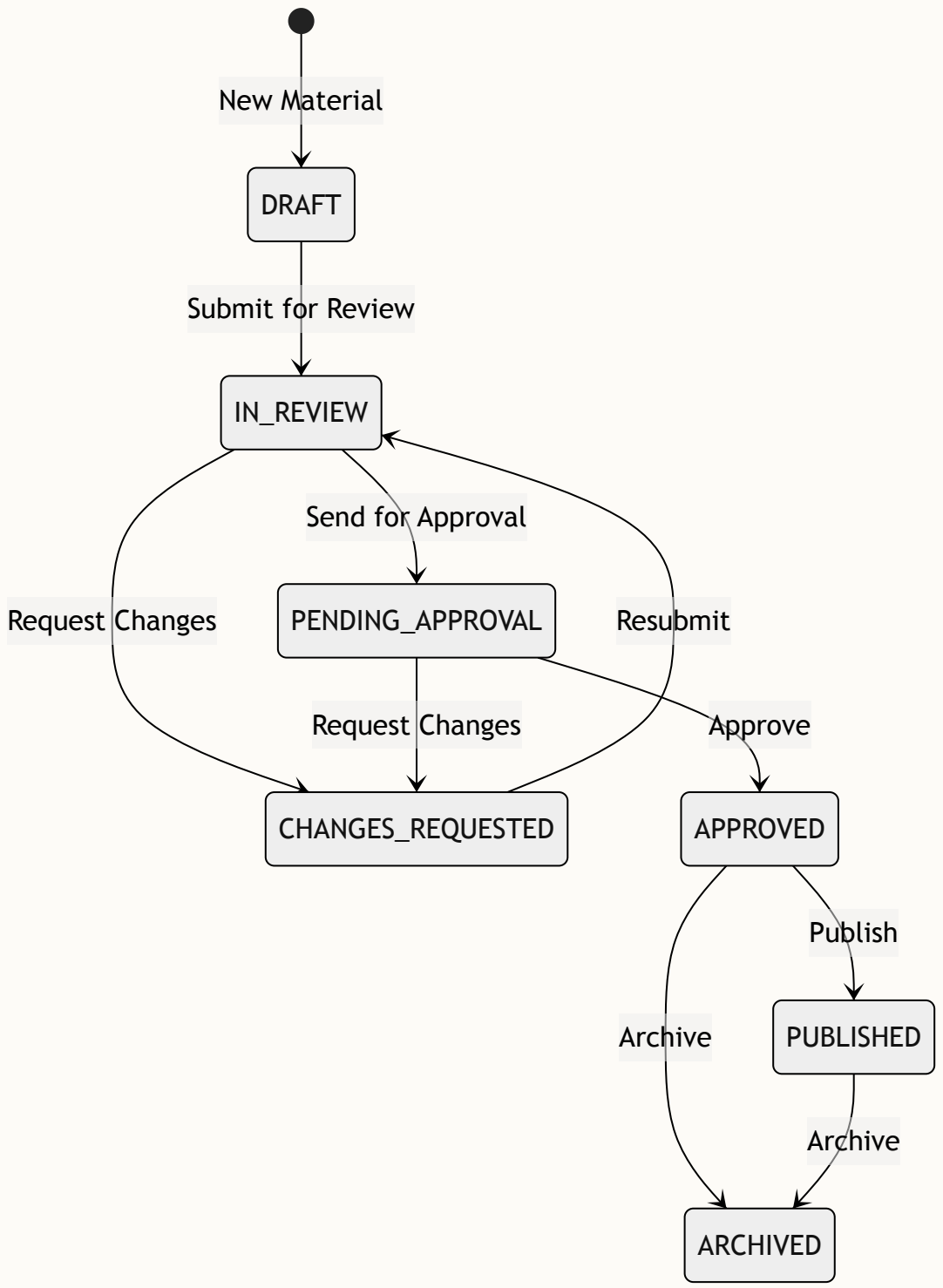
Role — أدوار المستخدمين

القيمة	الوصف
`OWNER`	مالك — صلاحيات كاملة
`ADMIN`	مدير — إدارة المستخدمين
`PROJECT_MANAGER`	مدير مشروع — إدارة المشاريع
`CREATOR`	منشئ — إنشاء وتحرير المواد
`REVIEWER`	مراجع — مراجعة وتعليق
`APPROVER`	معتمد — اعتماد نهائي

MaterialType — أنواع المواد

القيمة	الوصف	الأيقونة
`TEXT`	نصوص ومقالات	📄
`IMAGE`	صور وإنفوجرافيك	🖼️
`VIDEO`	فيديوهات	🎬
`AUDIO`	ملفات صوتية وبودكاست	🎙️

MaterialStatus — حالات المادة (دورة الحياة)



القيمة	اللون	الوصف
`DRAFT`	`#5B7DB1`	مسودة — قابلة للتعديل
`IN_REVIEW`	`#C4944A`	قيد المراجعة
`CHANGES_REQUESTED`	`#B8784A`	تعديلات مطلوبة
`PENDING_APPROVAL`	`#7B6BA5`	بانتظار الاعتماد النهائي
`APPROVED`	`#5B9A6B`	**معتمد (ممهور) ✓**
`PUBLISHED`	`#4A8A6B`	منشور
`ARCHIVED`	`#8B8B8B`	مؤرشف

3.3 القيود والفهارس

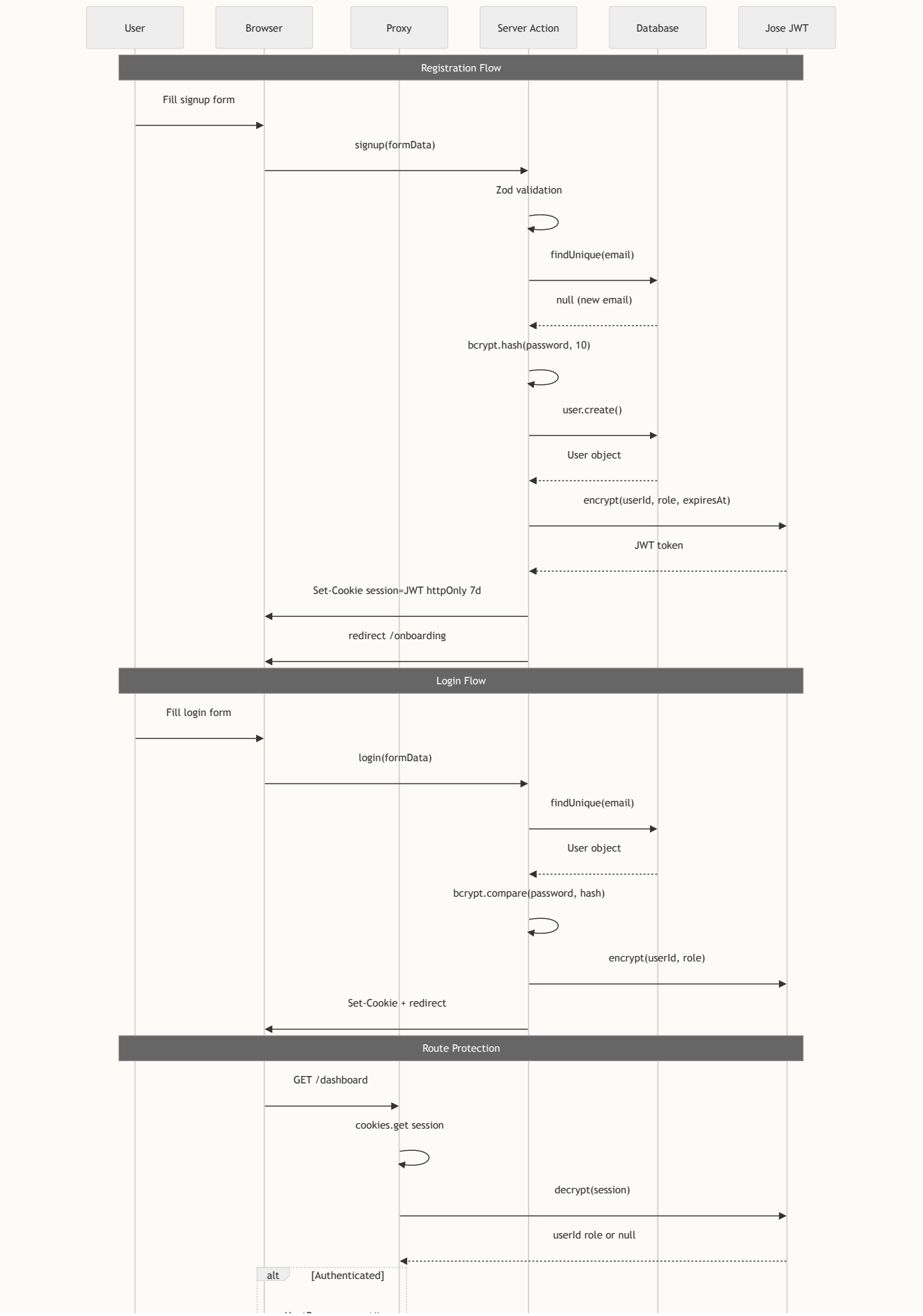
الجدول	القيد	الوصف
`User`	`email UNIQUE`	بريد إلكتروني فريد
`ProjectMember`	`@@unique([projectId, userId])`	عضو واحد لكل مشروع
`ProjectMember`	`onDelete: Cascade`	حذف الأعضاء عند حذف المشروع
`Material`	`onDelete: Cascade`	حذف المواد عند حذف المشروع
`Comment`	`onDelete: Cascade`	حذف التعليقات عند حذف المادة أو المستخدم
`StateTransition`	`onDelete: Cascade`	حذف السجل عند حذف المادة

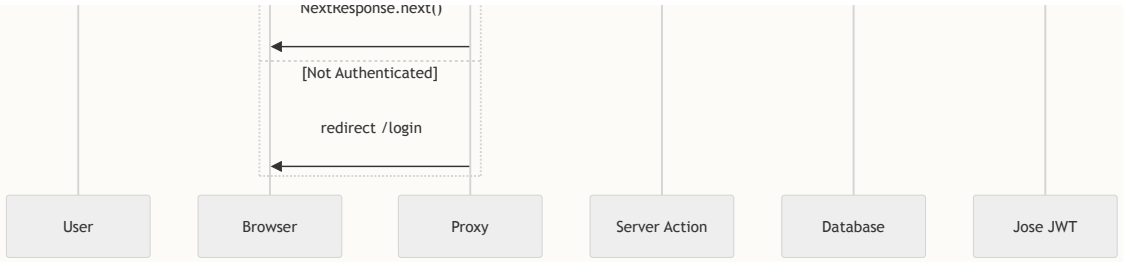
3.4 عميل Prisma

يستخدم المشروع **Prisma 7** مع `adapter-better-sqlite3` — وهو متطلب إلزامي في Prisma 7 حيث تم استبدال محرك الاستعلامات الثنائي بمحركات JavaScript. يتم تهيئة العميل كـ **Singleton** عبر `global.prisma` لتجنب إعادة الإنشاء في بيئة التطوير.

4. نظام المصادقة والأمان

4.1 معمارية المصادقة


User
Browser
Proxy
Server Action
Database
Jose JWT
Registration Flow
Fill signup form
signup(formData)
Zod validation
findUnique(email)
null (new email)
bcrypt.hash(password, 10)
user.create()
User object
encrypt(userId, role, expiresAt)
JWT token
Set-Cookie session=JWT httpOnly 7d
redirect /onboarding
Login Flow
Fill login form
login(formData)
findUnique(email)
User object
bcrypt.compare(password, hash)
encrypt(userId, role)
Set-Cookie + redirect
Route Protection
GET /dashboard
cookies.get session
decrypt(session)
userId role or null
alt
[Authenticated]



4.2 طبقات الأمان

الطبقة	الملف	الآلية
1. Proxy	`proxy.ts`	فحص JWT في الكوكيز قبل الوصول للصفحة
2. DAL	`dal.ts`	verifySession() — cached + server-only
3. Server Actions	`auth.ts`	التحقق من الجلسة + ownership check
4. Zod Validation	`definitions.ts`	مخططات تحقق صارمة
5. Password Hashing	bcryptjs	salt rounds = 10
6. JWT	jose	HS256, TTL = 7 أيام
7. Cookies	`session.ts`	httpOnly, secure (production), sameSite: lax

4.3 حماية المسارات

نوع المسار	المسارات	السلوك
محمي	dashboard/*, /onboarding/	غير مسجل → redirect /login
عام (auth)	login, /register/	مسجل → redirect /dashboard
عام	/, أي مسار آخر	متاح للجميع
مستثنى	/_next/, */api/*, ملفات ثابتة	لا فحص — تمرير مباشر

4.4 DAL — طبقة الوصول للبيانات

الدالة	الوصف	التخزين المؤقت
`verifySession()`	التحقق من الجلسة — يعيد {userId, role} أو redirect	React.cache()
`getCurrentUser()`	جلب بيانات المستخدم من DB	React.cache()
`getOptionalSession()`	تحقق اختياري — يعيد null بدل redirect	React.cache()

⚠️ مهم

جميع دوال DAL مُغلفة بـ React.cache() لتجنب الاستعلامات المكررة في نفس الطلب. كما تستخدم server-only لمنع الاستيراد في كود العميل.

5. هيكل المشروع (Architecture)

5.1 شجرة الملفات الكاملة

```
mamhoor/
├── .env                        # متغيرات البيئة
├── docker-compose.yml          # PostgreSQL (للإنتاج)
├── next.config.ts              # إعداد Next.js
├── prisma.config.ts            # إعداد Prisma
├── proxy.ts                    # حماية المسارات (Middleware)
├── tsconfig.json               # TypeScript
├── package.json                # التبعيات
│
├── prisma/
│   └── schema.prisma           # مخطط قاعدة البيانات (139 سطر)
│
└── src/
    ├── lib/                    # البنية التحتية
    │   ├── prisma.ts           #   عميل Prisma (Singleton + Adapter)
    │   ├── definitions.ts      #   مخططات Zod + أنواع TypeScript
    │   ├── session-crypto.ts   #   JWT encrypt/decrypt (Jose)
    │   ├── session.ts          #   إدارة الجلسات (Cookies)
    │   └── dal.ts              #   طبقة الوصول للبيانات (DAL)
    │
    └── app/
        ├── layout.tsx          # Root Layout (Cairo + Amiri, RTL)
        ├── globals.css         # نظام التصميم (298 سطر)
        ├── page.tsx            # صفحة الهبوط (Landing Page)
        │
        ├── actions/            # Server Actions
        │   ├── auth.ts         #   signup + login + logout
        │   ├── onboarding.ts   #   completeOnboarding
        │   └── material.ts     #   createMaterial
        │
        ├── onboarding/         # تهيئة المستخدم الجديد
        │   ├── page.tsx        #   Server Component
        │   ├── onboarding-wizard.tsx  # Client Wizard (3 خطوات)
        │   └── onboarding.module.css
        │
        ├── (auth)/             # صفحات المصادقة
        │   ├── layout.tsx
        │   ├── auth.module.css
        │   ├── login/page.tsx
        │   └── register/page.tsx
        │
        └── (dashboard)/        # لوحة التحكم
            ├── layout.tsx
            ├── dashboard-shell.tsx
```

```
        ├── dashboard.module.css
        └── dashboard/
            ├── page.tsx              # الرئيسية
            ├── projects/             # المشاريع
            ├── content/              # المواد
            ├── kanban/               # لوحة كانبان
            ├── comments/             # التعليقات
            ├── team/                 # الفريق
            ├── notifications/        # الإشعارات
            └── settings/             # الإعدادات
```

5.2 إحصائيات الشفرة المصدرية

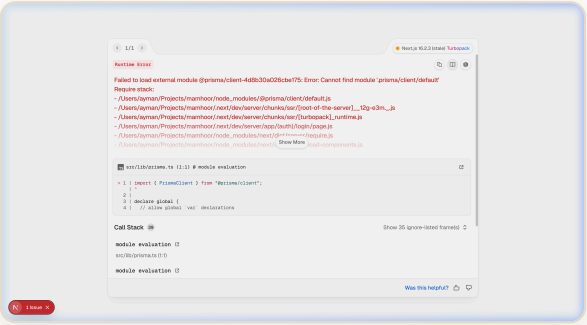
النوع	عدد الملفات	عدد الأسطر
TypeScript (.tsx)	23	~3,700
TypeScript (.ts)	8	~500
CSS Modules (.css)	13	~4,200
الإجمالي	**44**	**8,402**

5.3 نمط الـ Rendering

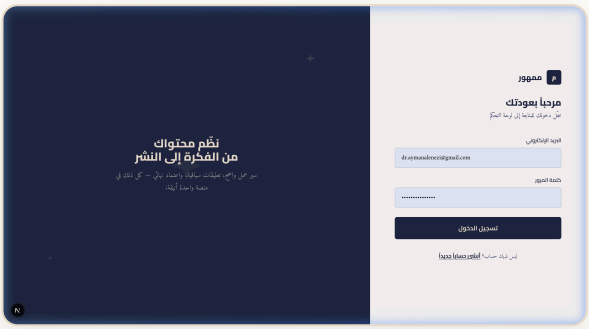
الصفحة	النمط	التفسير
/ Landing	SSG	Server Component — بدون بيانات ديناميكية
Login/Register	CSR	use client" + useActionState"
Onboarding	CSR	use client" + wizard state"
Dashboard Home	SSR	Server Component + Prisma queries
Projects List	SSR	Server Component + Prisma
Project Detail	SSR	Dynamic route + Prisma
Content List	SSR	Server Component + Prisma
Content Detail	SSR	Dynamic route + Prisma
New Content	Hybrid	Server (projects loader) + Client (form)
Kanban	SSG	Mock data (لم يُربط بعد)
Comments	SSG	Mock data (لم يُربط بعد)
Team	SSG	Mock data (لم يُربط بعد)
Notifications	SSG	Mock data (لم يُربط بعد)
Settings	SSR	Server Component + Prisma user data

6. واجهات المنصة — Screenshots

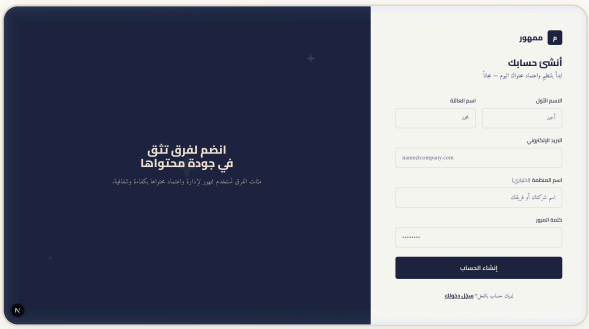
الصفحات العامة



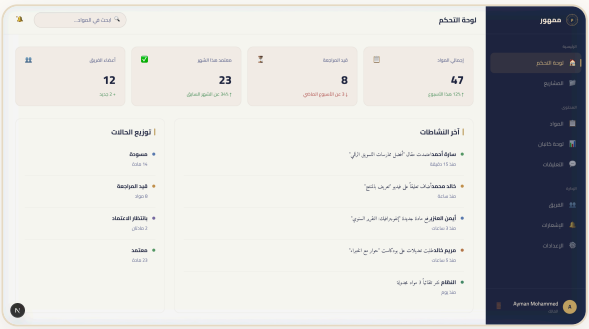
صفحة الهبوط الرئيسية



صفحة تسجيل الدخول



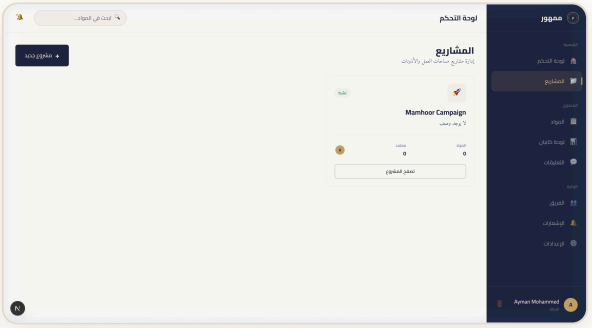
صفحة إنشاء حساب



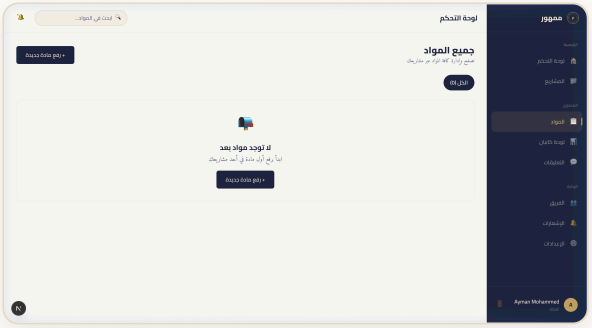
صفحة Onboarding — معالج 3 خطوات

لوحة التحكم (Dashboard)

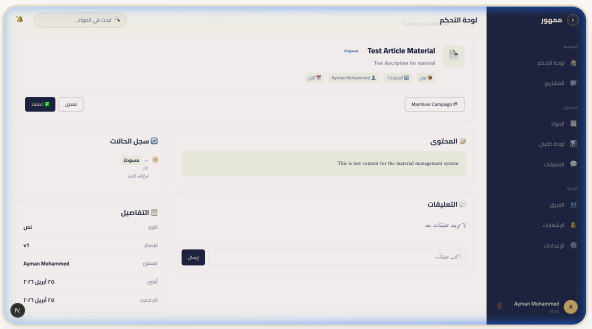
لوحة التحكم الرئيسية — إحصائيات حقيقية + آخر النشاطات + توزيع الحالات



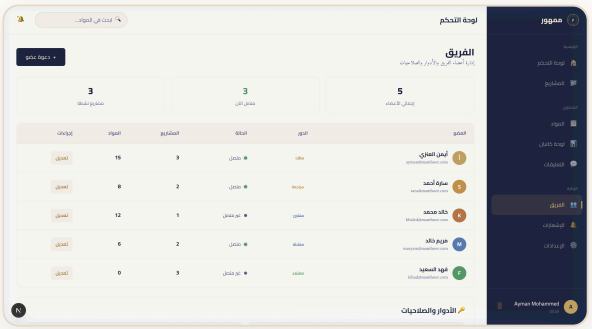
صفحة المشاريع — بطاقات حقيقية من DB



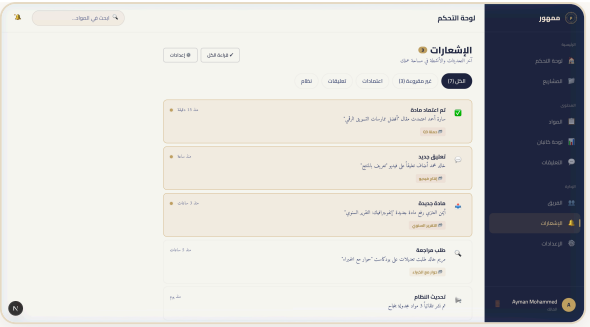
صفحة المواد — Empty state + فلاتر



تفاصيل مادة — محتوى + سجل حالات



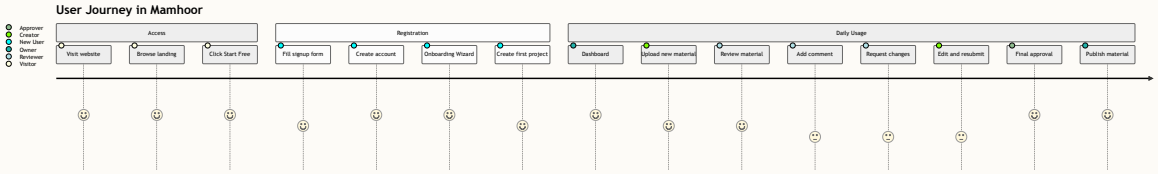
صفحة الفريق — الأعضاء والأدوار



صفحة الإشعارات

صفحة الإعدادات — بيانات حقيقية

7. رحلة المستخدم الكاملة (User Journey)



7.1 المرحلة 1 — الوصول والتسجيل

الخطوة	الصفحة	الإجراء	الملفات المعنية
1	/	زيارة صفحة الهبوط	page.tsx
2	register/	ملء نموذج التسجيل (الاسم، البريد، كلمة المرور)	register/page.tsx
3	Server	signup() — Zod validation → bcrypt hash → DB insert → JWT session	auth.ts
4	onboarding/	Redirect تلقائي (أول تسجيل)	proxy.ts

7.2 المرحلة 2 — التهيئة (Onboarding)

الخطوة	الوصف	الملفات
الخطوة 1	شاشة ترحيبية: "أهلاً {الاسم}! 👋"	onboarding-wizard.tsx
الخطوة 2	إنشاء المشروع الأول (اسم + وصف اختياري)	onboarding.ts
الخطوة 3	تأكيد النجاح + "انطلق إلى لوحة التحكم 🚀"	Client redirect

ما يحدث في الخلفية عند إتمام Onboarding:

1. `prisma.project.create()` — إنشاء المشروع مع members.create() (المالك كأول عضو)
2. `prisma.user.update()` — تعيين onboardingComplete = true
3. `redirect("/dashboard")` — توجيه للوحة التحكم

7.3 المرحلة 3 — الاستخدام اليومي

رفع مادة جديدة

الخطوة	الوصف
1	ينتقل إلى /dashboard/content/new
2	يختار نوع المادة (نص / صورة / فيديو / صوت)
3	يختار المشروع من القائمة المنسدلة (مشاريع حقيقية من DB)
4	يملأ العنوان والوصف والمحتوى
5	ينقر "رفع المادة"
6	createMaterial() — Zod validation → ownership check → material.create() → stateTransition.create(→DRAFT)
7	يتم التوجيه لـ /dashboard/content/{id}

دورة حياة المادة

الحالة	من يغيرها	الإجراء
DRAFT → IN_REVIEW	المنشئ	إرسال للمراجعة
IN_REVIEW → CHANGES_REQUESTED	المراجع	طلب تعديلات مع تعليقات
CHANGES_REQUESTED → IN_REVIEW	المنشئ	تعديل وإعادة إرسال
IN_REVIEW → PENDING_APPROVAL	المراجع	تمرير للاعتماد
PENDING_APPROVAL → APPROVED	المعتمد	✓ **ممهور!**
APPROVED → PUBLISHED	المالك/المدير	نشر

💡 **نصيحة**

كل تحول في الحالة يُسجَّل في جدول StateTransition مع fromStatus, toStatus, userId, note, g createdAt — مما يوفر سجل تدقيق كامل.

8. Server Actions

المنصة تحتوي على **3 Server Actions** تمثل العمليات الأساسية:

8.1 auth.ts — المصادقة

الدالة	المدخلات	المخرجات
`signup(state, formData)`	firstName, lastName, email, orgName, password	Success: redirect /onboarding — Error: field errors
`login(state, formData)`	email, password	Success: redirect /dashboard or /onboarding — Error: message
`logout()`	—	delete cookie → redirect /login

مخطط التحقق (Signup):

- الاسم الأول: ≤ 2 حروف
- اسم العائلة: ≤ 2 حروف
- البريد: بريد صالح
- كلمة المرور: ≤ 8 أحرف + حرف واحد + رقم واحد

8.2 onboarding.ts — التهيئة

الدالة	المدخلات	المخرجات
`completeOnboarding(formData)`	projectName, projectDesc	Success: redirect /dashboard — Error: {error: string}

8.3 material.ts — إدارة المواد

الدالة	المدخلات	المخرجات
`createMaterial(prevState, formData)`	title, description, type, projectId, content	Success: redirect /dashboard/content/{id} — Error: {error: string}

9. نظام التصميم (Design System)

9.1 توكنات CSS

الألوان الأساسية

المتغير	القيمة	الاستخدام
`--indigo`	#212842	اللون الرئيسي — خلفيات، نصوص
`--cream`	#FOE7D5	اللون الثانوي — خلفيات فاتحة
`--gold`	#C4A265	لون التمييز — أزرار، شارات
`--white`	#FDFBF7	خلفية الصفحة
`--black`	#12152B	أغمق درجة

الظلال (Shadows)

المتغير	القيمة
`--shadow-sm`	0 1px 3px rgba(33,40,66,0.06)
`--shadow-md`	0 4px 12px rgba(33,40,66,0.08)
`--shadow-lg`	0 8px 24px rgba(33,40,66,0.10)
`--shadow-xl`	0 16px 40px rgba(33,40,66,0.12)

الحركات (Animations) — 12 keyframes

`fadeIn` , `fadeInUp` , `fadeInDown` , `scaleIn` , `float` , `floatReverse` , `spin` , `stampPress` , `revealLine` , `shimmer` , `pulse` , `tiltCard`

الأزرار العامة

الفئة	النمط
`.btn--primary`	خلفية Indigo + نص Cream
`.btn--ghost`	شفاف + حدود
`.btn--cream`	خلفية Cream + نص Indigo
`.btn--gold`	خلفية Gold + نص أبيض
`.btn--sm`	حجم صغير (36px)
`.btn--lg`	حجم كبير (52px)

9.2 CSS Modules — التوزيع

الملف	الأسطر	النطاق
page.module.css	791	صفحة الهبوط
dashboard.module.css	607	هيكل الداشبورد
material-detail.module.css	450	تفاصيل المادة
auth.module.css	348	صفحات المصادقة
comments.module.css	343	صفحة التعليقات
project-detail.module.css	337	تفاصيل المشروع
globals.css	298	نظام التصميم العام
kanban.module.css	271	لوحة كانبان
team.module.css	269	صفحة الفريق
settings.module.css	269	الإعدادات
new-content.module.css	259	رفع مادة
onboarding.module.css	240	التهيئة
content.module.css	204	قائمة المواد
notifications.module.css	186	الإشعارات
projects.module.css	179	قائمة المشاريع

10. حالة ربط البيانات الحقيقية

الصفحة	الحالة	المصدر	التفاصيل
لوحة التحكم	✅ حقيقي	prisma.material.count, groupBy	إحصائيات + نشاطات + توزيع حالات
المشاريع	✅ حقيقي	prisma.project.findMany + _count	بطاقات + عدد مواد + أعضاء
تفاصيل المشروع	✅ حقيقي	prisma.project.findUnique	كامل
المواد	✅ حقيقي	prisma.material.findMany	جدول + فلاتر
تفاصيل المادة	✅ حقيقي	prisma.material.findUnique	كامل
رفع مادة	✅ حقيقي	Server Action createMaterial	CRUD كامل
الإعدادات	✅ حقيقي	prisma.user.findUnique	بيانات المستخدم
لوحة كانبان	⏳ Mock	بيانات تجريبية ثابتة	بحاجة لربط
التعليقات	⏳ Mock	بيانات تجريبية ثابتة	بحاجة لربط
الفريق	⏳ Mock	بيانات تجريبية ثابتة	بحاجة لربط
الإشعارات	⏳ Mock	بيانات تجريبية ثابتة	بحاجة لربط

11. المتغيرات البيئية

المتغير	القيمة الحالية	الوصف
`DATABASE_URL`	`file:./dev.db`	مسار قاعدة البيانات SQLite
SESSION_SECRET	`mamhoor-dev-secret-key-...`	مفتاح تشفير JWT

> ⚠️ **تحذير**
>
> يجب تغيير SESSION_SECRET عند النشر في بيئة الإنتاج. المفتاح الحالي هو مفتاح تطوير فقط.

12. البنية التحتية للحاويات (Docker)

يوجد ملف `docker-compose.yml` جاهز لبيئة الإنتاج مع **PostgreSQL 15**:

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: mamhoor
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mamhoordb
    ports: ["5432:5432"]
    volumes: [postgres_data:/var/lib/postgresql/data]
```

> 📝 **ملاحظة**
>
> **حالياً غير مُفعّل** — المشروع يستخدم SQLite. عند الانتقال لـ PostgreSQL:
>
> 1. تغيير provider في schema.prisma من sqlite إلى postgresql
> 2. تحديث DATABASE_URL في .env
> 3. استبدال @prisma/adapter-better-sqlite3 بـ @prisma/adapter-pg
> 4. تشغيل npx prisma migrate dev

13. التوصيات والخطوات المستقبلية

أولويات قصيرة المدى

- ربط **لوحة كانبان** بالبيانات الحقيقية مع Drag & Drop
- ربط **التعليقات** بـ Server Action (إضافة/حل)
- ربط **الفريق** بـ ProjectMember + دعوة أعضاء
- إنشاء Server Action لـ **مشروع جديد** من الداشبورد
- ربط **الإشعارات** بأحداث النظام الحقيقية

أولويات متوسطة المدى

- إضافة **رفع الملفات** (S3/Cloud Storage) للصور والفيديو والصوت
- Server Actions لـ **تغيير حالة المادة** (مراجعة ← اعتماد ← نشر)
- تفعيل **تعديل الملف الشخصي** في الإعدادات
- إضافة **نظام الإشعارات الفورية** (WebSocket أو SSE)
- إضافة **البحث** في المواد والمشاريع

أولويات طويلة المدى

- الانتقال من SQLite إلى **PostgreSQL** للإنتاج
- إضافة **اختبارات** (Unit + Integration + E2E)
- إعداد **CI/CD Pipeline**
- إضافة **دعم متعدد اللغات** (i18n)
- إضافة **سجل التدقيق** (Audit Log) المرئي
- إعداد **نظام النسخ الاحتياطي**
- **تحسين الأداء** — ISR, caching, connection pooling

14. كيفية التشغيل

```
# 1. تثبيت الاعتمادات
npm install

# 2. تهيئة قاعدة البيانات
npx prisma db push

# 3. تشغيل خادم التطوير
npm run dev

# 4. فتح المتصفح
open http://localhost:3000
```

نهاية التقرير — تم إعداده بتاريخ 25 أبريل 2026

© 2026 ممهور — جميع الحقوق محفوظة